1. Mantenimiento, actualizaciones y revisión de permisos (8.3)

Recomendación: Mantener una política de gestión de incidentes, actualizar periódicamente el software y las dependencias, y revisar los permisos de ficheros que contengan contraseñas o datos críticos y, si el peligro es inmediato, detener el proceso de la aplicación hasta que sea corregido.

¿Por qué?: Muchos usuarios casi nunca actualizan sus sistemas y tampoco realizan los mantenimientos con frecuencia. Es por eso que las vulnerabilidades conocidas siguen abiertas y suelen seguir un patrón, lo cual facilitaría la ocurrencia de estos incidentes.

2. Autenticación robusta y contraseñas bien protegidas (8.2 y 8.9)

Recomendación: Exigir contraseñas complejas, no guardarlas ni transmitirlas en texto claro y usar canales cifrados (HTTPS/TLS).

¿Por qué?: Por el simple hecho de que si establecemos credenciales con texto claro, contraseñas débiles, (man in the middle, replay..) etc. hay mayor probabilidad de que sean vulnerables. Las contraseñas en Base64 dentro de “config.properties” ya no están protegidas en estos casos, ya que Base64 es una codificación, no un cifrado y es fácil de revertir.

3. Validar siempre las entradas y evitar inyecciones (8.6 y 8.4)

Recomendación: Nunca confiar en lo que llega del cliente. Validar formato, longitud, rango y reglas de negocio, y usar consultas parametrizadas en lugar de concatenar texto en SQL.

¿Por qué?: Porque si no se valida correctamente, es una brecha muy común en aplicaciones web y, también, el origen de muchas otras vulnerabilidades como lo son inyecciones, desbordamientos, etc. El atacante puede vulnerar la información y la integridad de los datos, romper con el formato esperado o, incluso, romper con las reglas del negocio.

4. Criptografía correcta (8.9)

Recomendación:
No usar cifrados de flujo como RC4.
No usar hashes débiles: MD4, MD5 y SHA-1 tienen debilidades conocidas.
No dejar claves escritas dentro del código.
Generar tokens y números aleatorios con “java.security.SecureRandom.”
Cifrar los datos sensibles antes de transmitirlos o almacenarlos.

¿Por qué?: El cifrado protege sólo si está bien hecho, es decir, un pequeño fallo y lo deja inútil. Los algoritmos débiles se pueden romper, una clave escrita en el código la puede encontrar cualquiera que vea el programa y número aleatorio predecible permite a un atacante adivinar un token y se haga pasar por otra persona.

5. Manejo correcto de excepciones y uso de logs (8.7)

Recomendación: Usar un sistema de logs (como Log4j o SLF4J) en lugar de System.out.println o e.printStackTrace(). Manejar los errores con try/catch y mostrarle al usuario solo un mensaje simple, sin detalles técnicos.

¿Por qué?: Es recomendable y/o obligatorio en aplicaciones JEE utilizar logs y no salidas por consola ya que los logs te permiten depurar, guardar y trazar que es lo que sucedió en la aplicación dando lugar así a encontrar y corregir errores fácilmente.